

ESP32-C6 Wi-Fi / BLE Radio Blob OSI 実装実現性検討報告書

更新: 2026-10-06 (第 3 版: 書き手修正の段階)。独立監査 (job 7b9b10cc) は実施済みで、監査者が確認した範囲は **ローカル ESP-IDF v6.1 と Shizuku ソース** に限る。外部資料 (NuttX / esp-hal / Zephyr / docs サイト) は未確定のまま。本版は監査の差戻し (M1-M5) を反映した修正稿であり、**書き手は受入完了を主張しない**。再監査は別 session の上層が行う。対象: ESP32-C6 (HP single-core RV32IMAC)。
目的: FreeRTOS を起動しない Shizuku 上で、Espressif の Wi-Fi / BLE バイナリ blob が要求する OSI (OS 抽象) 関数表を実装できるかを、確定根拠と未確認事項を分けて検討する。
本書は静的な一次資料照合と現行ソース読解に基づく設計検討であり、実機・シミュレータでの blob 動作は一切確認していない。
実機を使った主張は含めない。

0. 前提・版・証拠水準

設計前提 (ユーザー指示): D1 (カーネルとカーネルオブジェクトの分離) は廃止済み。カーネルとカーネルオブジェクトの密結合は許容され、メモリ保護 (MMU/PMP) があれば重くても保護してよい。したがって「D1 に反する」ことは不可の理由にしない。
境界前提: Shizuku はチップ固有の SDK・toolchain を持ち、カーネル・ペリフェラル・イメージを担当する。XNO は SDK を持たない ISA 別 freestanding object の toolchain を担当する。XNO リポジトリは本作業で読んでいない (読取禁止)。
3 つの別の問い (本書では混ぜない):
1. **現行の公開 API / 実装の不足:** 今の Shizuku に何が無い。ソース行で確定できる。
2. **原理的可能性:** カーネル・カーネルオブジェクトを拡張すれば、互換 OS shim の意味論を実装できるか。D1 廃止下では「現在無い」から「原理的に不能」は導けない。
3. **実証:** blob が実際に動くか、時間制約を満たすか、保護が成立するか。本書では全て **未実証**。
証拠水準の記法:

- ・ **[確定-local]:** この worktree のソースを直接読んだ (file:line)。
- ・ **[確定-IDF]:** ローカルの ESP-IDF チェックアウト (.local/esp-idf、HEAD fff9895c82d744c7237be8847347bdd1b07c6643、git describe = v6.1) の原文を読んだ (file:line)。
- ・ **[二次抽出]:** WebFetch (要約モデルを介した取得) で得た。項目名・件数・行番号は原文と食い違う可能性がある。実際、1 回目の取得は「160 項目」と述べたが、同じ応答の一覧を数え直すと 130 行であり、取得結果の自己申告件数は信頼できなかった (件数は本書で数え直した)。
- ・ **[未確認]:** 確認できていない。推測で埋めない。

版情報 (確認したものと未確認のものとの区別):

対象	確認した版 / ref	備考
ESP-IDF	v6.1 = commit fff9895c82d744c7237be8847347bdd1b07c6643 (ローカル .local/esp-idf の HEAD。git describe が v6.1)	SHA 固定は確定。 docs/06_esp32_port_manifest.typ:9,87,117 は fff9895c82d744c7237be8847347bdd1b07c6643 を固定対象と書いており、 本書の検討対象 v6.1 と版が異なる。 v6.1.1 での項目数・初期化子は 未照合 。「最新」かどうか未確認。
NuttX	ブランチ master の取得時点	SHA 未確認。行番号は取得時点のもので固定されない。
Zephyr hal_espressif	検索結果の一覧のみ (二次)	中身は読んでいない。[未確認]。
esp-radio	docs 上で 1.0.0-beta.2 と 0.18.0 の 2 版を確認	1.0.0-beta.2 が「現公開」であることはユーザー指示による。 最新であるとは本書では主張しない。
esp-radio-rtos-driver	docs.rs 表示 0.4.2	公開日の表示 (2026-09-16) は二次抽出。

取得経路と証拠の層: IDF は **ローカルチェックアウトの原文** (確定-IDF) を読んだ。NuttX / esp-hal / esp-radio docs / docs.rs / Zephyr は WebFetch (要約モデル経由) が検索結果で、sandbox から raw.githubusercontent.com と api.github.com への直接接続は拒否されたため、**これらの SHA は固定できず、行番号は全て二次抽出で未確定のまま**。IDF 以外の項目は再確認が必要。

1. 総合判定

問い	判定	根拠と限界
現行 Shizuku (今のソース) だけで OSI 表を埋められるか	不可 (確定-local)	C6 の arch / board backend は未完成で、本番ビルドは #error で拒否される (internal_headers/shizuku/archs/rv32_c6.hpp:14,20、boards/esp32_c6.hpp:14,19)。trap entry・SYSTIMER 割込み・割込みマトリクスは未実装。加えて、ブロック/起床・優先度・ISR からの通知・ソフトウェアタイマの各機構が公開されていない (第 5 節)。
カーネル/カーネルオブジェクトを拡張すれば、OSI が要求する意味論を原理的に実装できるか	原理的な不能は示されていない (unknown)	本書は不能を示す根拠を持たない。ブロック/起床、優先度、タイマ、キューは方針側 (カーネルオブジェクト) に実装できる設計である (templates/thread.hpp:77-78 は優先度と起床時刻を「方針としてカーネルオブジェクトが持つ」とする)。ただし 実装可能であることと、blob が要求する時間制約を満たすことは別 で、後者は未実証。
FreeRTOS を起動せずに Wi-Fi/BLE blob をリンクして動かせるか	unknown (実証不能)	OSI 関数表は関数ポインタで渡す形なので、表の実装を差し替えること自体は構造上可能 (一次資料で表の存在を確認)。しかし IDF 側の初期化経路 (PHY・clock・割込み割当・NVS 等) を FreeRTOS 無しでどこまで再利用できるかは未確認 (第 9 節)。

問い	判定	根拠と限界
M-mode trusted blob の保護境界	2 つに分離して unknown	(a) blob を信頼境界内 (TCB) に置くと決めるなら、blob の CPU アクセスは境界内であり、その点では保護問題は生じない。(b) DMA / APM でバスマスタの到達範囲を制限できるか、その policy が実機で効くかは別問題で 未実証・未確認 (第 7 節)。
OSI object を XNO に置く妥当性	部分的に妥当 (設計判断、未実証)	チップ非依存の OS 意味論 (キュー、セマフォ等) と、チップ固有 API (PHY・clock・割込み・modem) は分離でき、前者のみ XNO 候補。blob 本体と OSI 表の配置は Shizuku 側 (第 8 節)。

2. 一次資料一覧 (表の実体と出典)

対象	名称 / 版	出典 (IDF = ローカル原文 [確定-IDF]、それ以外の行番号は二次抽出)
Wi-Fi OSI 表 (IDF)	wifi_osi_funcs_t、_version = 0x00000009、_magic = 0xDEADBEAF	components/esp_wifi/include/esp_private/wifi_os_adapter.h (v6.1 = fff9895)。ESP_WIFI_OS_ADAPTER_VERSION / _MAGIC は 行 20-21 [確定-IDF]。C6 の初期化子は components/esp_wifi/esp32c6/esp_adapter.c:628-758 に g_wifi_osi_funcs として存在し、126 エントリ (第 3 節)。
Wi-Fi OSI 表の実体 (NuttX)	g_wifi_osi_funcs、初期化子 127 項目	arch/risc-v/src/esp32c6/esp_wifi_adapter.c の行 720-847 (master 取得時点、二次抽出)。旧版が示した 315-437 は取得結果と一致しない。
Wi-Fi OSI 表の実体 (Zephyr)	esp_wifi_adapter.c (hal_espressif)	codex.ac6.fr のクロスリファレンス (検索結果のみ。中身は未読。blob 取得に west blobs fetch hal_espressif が要するという記述のみ確認)。
Wi-Fi OSI 表の実体 (Rust)	esp-radio の src/wifi/os_adapter/ (ディレクトリ名のみ確認)	esp-hal リポジトリ esp-radio/src/wifi。表の初期化子ファイルまでは未確認。旧文書の internal:: _ESP_RADIO_G_WIFI_OSI_FUNCS という名は確認できていないため削除した。
BLE NPL 表	struct npl_funcs_t (45 メンバ)	nimble_npl_os.h と npl_os_freertos.c (両方で 45 を数えた。npl_funcs ポインタは動的確保の複製を指す)。
BLE コントローラ拡張表	struct ext_funcs_t (ext_version=0x20250825、magic=0xA5A5A5A5) と struct osi_coex_funcs_t	components/bt/controller/esp32c6/bt.c [確定-IDF]: osi_coex_funcs_t 87-94、ext_funcs_t 96-114 (EXT_FUNC_VERSION/MAGIC は 79-80)、s_osi_coex_funcs_ro 523-530、ext_funcs_ro 532-548、esp_register_ext_funcs 呼出し 1091、npl_freertos_funcs_init 1098、esp_register_npl_funcs 1105、ble_osi_coex_funcs_register 1126。旧文書はこれらの表を欠き、行番号も 1086/1088 と誤っていた。
esp-radio 要件	1.0.0-beta.2 / 0.18.0	1.0.0-beta.2 docs と 0.18.0 docs (C6)。
RTOS 差し替え契約	esp-radio-rtos-driver 0.4.2	docs.rs (二次抽出)。

3. Wi-Fi OSI 関数表 (wifi_osi_funcs_t) の項目と Shizuku 対応

件数の数え方 [確定-IDF。ローカル wifi_os_adapter.h を数えた]: ヘッダの構造体は全体で 130 フィールド (条件付き #if を含む)。C6 構成では、ソース上の次の条件により 3 項目が有効にならず、C6 の有効フィールドは 127 (_version + _magic + 関数ポインタ 125)。

- ・ _phy_common_clock_enable/disable (2): #if CONFIG_IDF_TARGET_ESP32 || CONFIG_IDF_TARGET_ESP32S2 || CONFIG_ESP_WIFI_TARGET_ESP32 の内側 (C6 ではこの条件が偽)。
- ・ _coex_configure_preemption_end_cb (1): #if CONFIG_IDF_TARGET_ESP32S31 の内側 (C6 では偽)。

ヘッダの項目数と初期化子の件数は別物。C6 の実体 esp_adapter.c:628-758 の g_wifi_osi_funcs 初期化子は **126 エントリ** で、ヘッダにある _coex_condition_set が **初期化されない (NULL)**。したがって「ヘッダ 127 = 全項目が初期化済み」ではない。blob が NULL の _coex_condition_set を呼ぶのか、呼ばれない設計なのかは **未知**。_regdma_link_set_write_wait_content _sleep_retention_find_link_by_id と _wifi_{bb,mac}_sleep_retention_{attach,detach} の初期化は SOC_PM_MODEM_RETENTION_BY_REGDMA 条件下 (esp_adapter.c:742 付近)で、C6 では soc_caps.h:482 が (1)。**NuttX master の 127 項目という二次抽出が IDF の件数と一致しても、IDF が全項目を初期化していることは含意しない** (NuttX の件数は参考情報に降格)。IDF の版が違えば件数も違ひ得て、**v6.1.1 は未照合**。旧文書の一覧は 125 項目の多くを省略していた。下表は C6 構成の関数ポインタ 125 を全て数える。

凡例: 「現状」=今の Shizuku (確定-local)。「実装の手段」=D1 廃止下で原理的に何が必要か (実証なし)。

分類	項目 (wifi_osi_funcs_t メンバ。先頭 _ を含む)	数	現状 (確定-local)	実装の手段 / 未確認
割込み	_set_intr_clear_intr_set_isr_ints_on _ints_off_is_from_isr	9	未対応 。C6 の trap entry・SYSTIMER・割	割込み割当・ISR 登録は chip 固有。mstatus.MIE での単核割

分類	項目 (wifi_osi_funcs_t メンバ。先頭_を含む)	数	現状 (確定-local)	実装の手段 / 未確認
	_wifi_int_disable_wifi_int_restore _task_yield_from_isr		込みマトリクスが未実装 (rv32_c6.hpp:14、esp32_c6.hpp:14)。外部割込みからスレッドへ通知する公開口が無い。	込み禁止は実装可 (未実装)。 _is_from_isr は trap 深度の公開が必要。 _task_yield_from_isr は ISR 出口での切替要求で、機構追加が要る。
スピンロック	_spin_lock_create _spin_lock_delete	2	公開口なし (内部に CAS ロック table_lock、handler.cpp:980)。	単核なので割込み禁止で足りる設計が可能 (未実証)。
セマフォ	_semphr_create _semphr_delete _semphr_take _semphr_give _wifi_thread_semphr_get	5	公開 API なし。 待機スレッドを眠らせて起こす機構が無い (第 5 節)。	カーネルオブジェクト側に待機リストと起床を実装すれば原理的に可能。 _wifi_thread_semphr_get はスレッド毎セマフォ (スレッド局所の記憶) が要る。ISR からの give は割込み経路の整備後。
ミューテックス	_mutex_create _recursive_mutex_create _mutex_delete _mutex_lock _mutex_unlock	5	公開 API なし。	所有者・再帰カウント・(必要なら) 優先度継承は方針側で実装可能。優先度継承が blob に必要かは未確認。
キュー	_queue_create _queue_delete _queue_send _queue_send_from_isr _queue_send_to_back _queue_send_to_front _queue_rcv _queue_msg_waiting_wifi_create_queue _wifi_delete_queue	10	公開 API なし。 IDF 側の定数は OSI_QUEUE_SEND_FRONT/ BACK/OVERWRITE = 0/1/2、無期限待ち OSI_FUNCS_TIME_BLOCKING = 0xffffffff (wifi_os_adapter.h:23-27) [確定-IDF] で、上書き送信と無期限待ちの語彙が要る。STREAM は SPSC の制御プレーン (object_api.hpp:94-102) で、多対一・タイムアウト付き受信・先頭挿入の語彙は無い。	方針側で実装可能 (可変長メッセージ、待機リスト)。ISR 送信は割込み経路の整備後。
イベントグループ	_event_group_create _event_group_delete _event_group_set_bits _event_group_clear_bits _event_group_wait_bits	5	公開 API なし。	待機条件 (全ビット/任意ビット、クリア有無) を持つ待機リストが要る。実装可能 (未実証)。
タスク	_task_create_pinned_to_core _task_create_task_delete_task_delay _task_ms_to_tick_task_get_current_task _task_get_max_priority	7	一部対応。 SPAWN (object_api.hpp:46)、EXIT_THREAD (:60)、KILL_THREAD (:123)、SLEEP_US (:65)、GET_CURRENT_OBJECT (:40) がある。優先度は無い (templates/thread.hpp:77-78、handler.cpp:713-824 は回転子によるラウンドロビン)。	_task_create は渡された関数・スタック深さ・優先度・コア指定を spawn_request (templates/kernel.hpp:144-164) の引数に写せる。優先度は方針側の追加が必要。 _task_get_max_priority は実装する優先度の段数に依存。コア指定は単核なので意味が縮退。
メモリ	_malloc_free_get_free_heap_size _malloc_internal_realloc_internal _calloc_internal_zalloc_internal _wifi_malloc_wifi_realloc_wifi_calloc _wifi_zalloc	11	一部対応。 MEMORY_ALLOCATE/RELEASE/HAND_OVER/OWNER (object_api.hpp:75-78)、24 階級フリーリスト (templates/	realloc/calloc/zalloc は shim で可。「internal / DMA 可能」の領域区分は、blob が実際に要求する領域条件 (一次資料で未確認) に依存し、C6 の SRAM 区分を調べる必要がある。

分類	項目 (wifi_osi_funcs_t メンバ。先頭_を含む)	数	現状 (確定-local)	実装の手段 / 未確認
			kernel_object.hpp:111-138)。 realloc/calloc/zalloc/ 空き容量報告は無い。 DMA 可能領域の区別は arena に見当たらない (memory.cpp の grep で DMA の語は 0 件)。	
タイマ / 時刻	_timer_arm_timer_disarm_timer_done _timer_setfn_timer_arm_us _esp_timer_get_time_get_time	7	未対応 。コールバックを 時限起動するタイマが 無い。SLEEP_US は呼出 しスレッド自身の待機 のみ (handler.cpp:827-862)。 SYSTIMER は C6 で未 実装。	タイマ機構はカーネルオブ ジェクト側にタイマ表と起床 経路を足せば原理的に可能。 コールバックの実行文脈 (タイ マ用スレッドか割込み文脈か) は blob の要求次第で未確認。
HW / PHY / clock / PM	_env_is_chip _dport_access_stall_other_cpu_start_wrap _dport_access_stall_other_cpu_end_wrap _wifi_pm_sleep_lock_acquire _wifi_pm_sleep_lock_release _phy_disable_phy_enable _phy_update_country_info_read_mac _wifi_reset_mac_wifi_clock_enable _wifi_clock_disable _wifi_rtc_enable_iso _wifi_rtc_disable_iso_slowclk_cal_get	15	chip 固有 (Shizuku 側 の担当)。C6 の modem / PMU / RTC 操作は未実装。	_read_mac は eFuse 読出し。 PHY・clock・isolation・較 正は IDF の esp_phy / esp_clk / esp_pm 相当の再実装 か再利用が要る。再利用可能 性は第 9 節。要検証。
Coex	_coex_init_coex_deinit_coex_enable _coex_disable_coex_status_get _coex_condition_set_coex_wifi_request _coex_wifi_release _coex_wifi_channel_set _coex_event_duration_get_coex_pti_get _coex_schm_status_bit_clear _coex_schm_status_bit_set _coex_schm_interval_set _coex_schm_interval_get _coex_schm_curr_period_get _coex_schm_curr_phase_get _coex_schm_process_restart _coex_schm_register_cb _coex_register_start_cb _coex_schm_flexible_period_set _coex_schm_flexible_period_get _coex_schm_get_phase_by_idx	23	Shizuku に対応物なし。 旧文書の「21 項目」は 誤り (C6 構成で 23)。 このうち _coex_condition_set は C6 の初期化子で NULL (esp_adapter.c:628-758)。 blob が呼ぶかは未知。	これらは OS 機能ではなく IDF 側 coex ライブラリ (esp- coex-lib、Wi-Fi/BT 時分割) への橋渡しで、blob 側に実体 がある可能性が高いが、本書 では未確認。要検証。
スリープ保持	_regdma_link_set_write_wait_content _sleep_retention_find_link_by_id _wifi_bb_sleep_retention_attach _wifi_bb_sleep_retention_detach _wifi_mac_sleep_retention_attach _wifi_mac_sleep_retention_detach	6	未対応 (C6 固有、条件 付き項目)。IDF 側の初 期化子は SOC_PM_MODEM_RETENTION_BY_REGDMA 条件で、C6 では soc_caps.h:482 = 1 (esp_adapter.c:742 付 近)。	省電力を使わない構成でスタ ブ化できるか、blob が必須呼 出しをするかは 未検証 。
Wi-Fi 6	_wifi_disable_ac_ax	1	Shizuku 側に対応物な し。条件は CONFIG_SOC_WIFI_HE_SUPPORT (wifi_os_adapter.h [確 定-IDF])。	戻り値の意味を一次資料で確 認していない。「固定定数で足 る」とは書かない。要検証。
NVS	_nvs_set_i8_nvs_get_i8_nvs_set_u8 _nvs_get_u8_nvs_set_u16_nvs_get_u16 _nvs_open_nvs_close_nvs_commit _nvs_set_blob_nvs_get_blob _nvs_erase_key	12	Shizuku に永続ストア なし。	RAM 上の KV で代用できるか (校正データの永続性が要るか) は未確認。要検証。

分類	項目 (wifi_osi_funcs_t メンバ。先頭_を含む)	数	現状 (確定-local)	実装の手段 / 未確認
その他	_event_post _rand _get_random _random _log_write _log_writew _log_timestamp	7	ログ出力・乱数・イベント通知の公開なし (C6 の UART/USB 診断は未実装)。	乱数は C6 の RNG レジスタ、ログは出力経路に依存。 _event_post は IDF のイベントループ先で、Shizuku 側の受け口の設計が要る。
合計		125		1. _version + _magic = 127 フィールド (C6 構成)

4. BLE: npl_funcs_t (45) と ext_funcs_t (C6 で 13 フィールド)

struct npl_funcs_t (nimble_npl_os.h と npl_os_freertos.c の両方で 45 メンバを数えた。順序は両方で異なる)。
p_ble_npl_hw_set_isr は **構造体では無条件** のメンバ (nimble_npl_os.h:120) だが、**初期化子側** は #if NIMBLE_CFG_CONTROLLER || CONFIG_NIMBLE_CONTROLLER_MODE の中で NULL を設定する (npl_os_freertos.c:1093-1094)。インラインラップ ble_npl_hw_set_isr は #if NIMBLE_CFG_CONTROLLER 内 (nimble_npl_os.h:349-355)。条件が成り立つ構成では NULL であり、blob がこれと呼ぶかは未知。

分類	項目 (p_ble_npl_ を省略)	数	Shizuku 側の状況
OS 状態	os_started get_current_task_id	2	GET_CURRENT_OBJECT はあるが、不透明タスクハンドルの契約は未定義。実装は容易だが契約合わせは要確認。
イベントキュー	eventq_init eventq_deinit eventq_get eventq_put eventq_put_to_front eventq_remove eventq_is_empty	7	公開 API なし。eventq_get の待機とタイムアウトが必要。
イベント	event_run event_init event_deinit event_reset event_is_queued event_get_arg event_set_arg	7	データ構造と呼出しのみ。OS 機構は不要。
ミューテックス	mutex_init mutex_deinit mutex_pend mutex_release	4	公開 API なし。
セマフォ	sem_init sem_deinit sem_pend sem_release sem_get_count	5	公開 API なし。
コールアウト	callout_init callout_reset callout_stop callout_deinit callout_mem_reset callout_is_active callout_get_ticks callout_remaining_ticks callout_set_arg	9	タイマ機構が無い。イベントキューへの投入が要る。
時間	time_get time_ms_to_ticks time_ticks_to_ms time_ms_to_ticks32 time_ticks_to_ms32 time_delay get_time_forever	7	tick 換算は純計算。時刻源は C6 の SYSTIMER で未実装。time_delay は SLEEP_US で代用できる余地があるが、待機中の起床契約は要確認。
HW / 臨界区間	hw_set_isr (初期化子は NULL。上記) hw_enter_critical hw_exit_critical hw_is_in_critical	4	割り込み経路が未実装。
合計		45	

struct ext_funcs_t (bt.c:96-114、実体 ext_funcs_ro 532-548、登録 1091 [確定-IDF])。旧文書は BLE 節でこれを欠いていた。C6 では ext_version、関数ポインタ 11 個、magic の計 13 フィールドと読める (#if CONFIG_IDF_TARGET_ESP32C6 の _esp_reset_modem を含む)。

メンバ	Shizuku 側の含意
_esp_intr_alloc _esp_intr_free	割り込み割り当 (chip 固有)。C6 の割り込みマトリクスは未実装。Shizuku 側で提供する。
_malloc _free	アロケータ。要求領域 (internal/DMA 可) は一次資料で未確認。
_task_create _task_delete	BLE コントローラ用タスクの生成。SPAWN に写せるが、優先度・スタック・コア指定の意味合わせが要る。
_osi_assert _os_random	assert とログ、乱数。
_ecc_gen_key_pair _ecc_gen_dh_key	ECC 鍵生成と DH (暗号処理)。実装・移植元は IDF の関数 (esp_ecc_*) に依存し、再利用の可否は未確認。
_esp_reset_modem	C6 固有の modem リセット。chip 固有。

struct osi_coex_funcs_t (bt.c:87-94、実体 s_osi_coex_funcs_ro 523-530、ble_osi_coex_funcs_register 呼出し 1126 [確定-IDF])。BLE 用の別表で、旧文書にも前稿にも無かった。_magic、_version に関数ポインタ 4 つ。_coex_wifi_sleep_set と _coex_core_ble_conn_dyn_prio_get は NULL で、_coex_schm_status_bit_set/clear のみラップを持つ。blob が NULL の 2 つを呼ぶかは **未知**。

BLE 初期化は表の登録だけではない (bt.c:1091-1145 [確定-IDF])。esp_register_ext_funcs(1091)、npl_freertos_funcs_init(1098)、esp_register_npl_funcs(1105)に加え、IDF 内部の modem_clock_module_enable(PERIPH_BT_MODULE) (1120)、esp_phy_modem_init()(1124)、ble_osi_coex_funcs_register(1126)、coex_init()(1133)、esp_ble_register_bb_funcs() (1143) を呼ぶ。これらが FreeRTOS 無しで再利用できるかは **一次資料上は未確認** で、OSI 表を埋めるだけでは BLE は起動しないことの根拠になる。

5. 現行 Shizuku のスレッド・同期・タイマ機構 (確定-local)

以下は source/kernel/*.cpp、source/kernel_object/handler.cpp、internal_headers/shizuku/templates/*.hpp を読んだ結果。「**現在無い**」ことを示すもので、「**実装できない**」ことは示さない。

1. **カーネルのプリミティブは 4 つ**: CALL RETURN SWITCH GRANT (kernel_abi.hpp:28-44)。GRANT は期限付きの実行権貸与で、期限が来ると強制回収する (templates/kernel.hpp:259 (宣言)、source/kernel/thread.cpp:351 (定義) timer_expired、source/kernel/dispatch.cpp:347 pendsv_dispatch)。この強制回収は時間駆動の取り上げであり、優先度や外部事象による即時のプリエンプトではない。旧文書の「協調のみ」は不正確 (期限による強制はある) で、「優先度に基づく即時プリエンプトは無い」と書くのが正確。
 2. **スケジューラは方針側**: schedule() は回転子から順に READY のスレッドを探し、GRANT か SWITCH で渡す (handler.cpp:713-824)。優先度の概念は無い。優先度や起床時刻はカーネルの thread に無く、カーネルオブジェクトが表で持つ (templates/thread.hpp:77-78)。
 3. **待機は「寝る」だけ**: sleep_us は締切までループし、その間 schedule(self) を呼んで他へ譲る (handler.cpp:827-862、ループは約 850-858 行)。待機スレッドを待機リストに載せて外部事象で起こす機構は無い。起床時刻の表 m_wake_at を schedule() が見て眠っている相手を飛ばす (handler.cpp:805) のみ。
 4. **ISR からスレッドへの通知は無い**: C6 の trap entry・SYSTIMER・割り込みマトリクスは未実装で、本番ビルドは #error になる (rv32_c6.hpp:14,20、boards/esp32_c6.hpp:14,19)。
 5. **同期プリミティブは公開されていない**: 内部の table_lock (CAS スピン、handler.cpp:980) のみ。object_api.hpp にセマフォ・ミューテックス・キュー・イベントグループ・タイマの番号は無い (object_api.hpp:15-138)。
 6. **スレッドの記憶**: スタックと台帳は呼び側が用意して貸す (templates/kernel.hpp:51-60,142)。SPAWN は戻りを待たない起動。
- 結論**: 上記は全て「現状のプリミティブ不足」であり、D1 廃止下では、これらの方針側へ追加することを妨げる原理は示されていない。ただし追加した機構が blob の時間制約を満たすかは未実証。

6. 「no_std は OS 不要を意味しない」(esp-radio の記述に基づく)

esp-radio の公式 docs は次のように述べる。

- ・ 0.18.0: 動的メモリアロケータと **preemptive なタスクスケジューラ** がアプリケーションに必要で、最も簡単な選択肢は esp-rtos。「スケジューラは 802.15.4 には不要」(二次抽出)。
- ・ 1.0.0-beta.2: Wi-Fi と BLE は動的メモリアロケータと preemptive なタスクスケジューラを実行時に必要とし、esp-alloc と esp-rtos が推奨だが、必要なインタフェースを実装する任意のアロケータ/RTOS で足りる。無線の初期化前に esp_rtos::start(...) でスケジューラを起動する (二次抽出)。
- ・ 契約は別 crate esp-radio-rtos-driver 0.4.2 が定義する。実装すべき trait は SchedulerImplementation、SemaphoreImplementation、QueueImplementation、TimerImplementation。待機キュー実装 (WaitQueueImplementation) を与えれば CompatSemaphore / CompatTimer / CompatQueue が既定実装として使える。要求される機能にはタスク生成・現在タスク・優先度・μs 精度の時刻 (now, usleep)・割り込み文脈/通常文脈の yield が含まれる (二次抽出)。

ここから言えること: (1) Rust で no_std であることは OS 不要を意味しない。docs 自体が preemptive なスケジューラを要件としている。(2) OSI 表の意味論は、Rust 側でも OS 提供者が実装する契約として分離されている。(3) 要件は **公式 docs の記述** であり、blob が実際に必要とする最小限を測った結果ではない。

旧文書から削除した主張: 旧文書は esp-wifi の src/preempt/、src/timer/、src/compat/ を引いて内部構造を断定したが、現行の esp-radio では RTOS 部分が別 crate (esp-rtos / esp-radio-rtos-driver) に分かれており、旧パス・内部構造の記述は今回の取得で確認できなかったため削除した。「FreeRTOS クローンを抱え込む」「blob を騙している」といった表現も根拠のない断定として削除した。

7. M-mode trusted blob と保護境界

区別すべき 2 つの問い:

1. **信頼境界 (TCB) の問題**: blob を M-mode の trusted object として置く場合、blob はカーネルと同じ信頼境界に入る。blob の CPU 側コードの誤りはカーネルを破壊し得るため、署名・出所の信頼が前提になる。これは設計上の取り決めで、「保護が成立しない」ことの証明ではない。docs/06_esp32_port_manifest.typ も「カーネル空間 object は署名済み/信頼済みコードに限定し、保護付き object と明確に区別する」と書く。
 2. **バスマスタ (DMA 等) の到達範囲**: RISC-V PMP は CPU のアクセスを検査する。CPU 以外のバスマスタは別の機構で制御される。**ローカル IDF v6.1 の一次資料で確定できること** (components/esp-tee/subproject/main/soc/esp32c6/esp-tee_apm_prot_cfg.c、components/soc/esp32c6/include/soc/apm_defs.h): ESP32-C6 には APM (HP_APM、領域数 APM_CTRL_HP_APM_REGION_NUM = 16、apm_defs.h:20) がある。esp-tee_apm_prot_cfg.c のコメントにあるマスタ ID 表 (53-77 行) には HP CPU、LP CPU、SDIO_SLV、MEM_MONITOR、TRACE、SPI2、UHCI、I2S、AES、SHA、ADC、PARLIO 等があり、CPU 以外のマスタが存在することは確定。**領域設定はアクセス経路 APM_CTRL_ACCESS_PATH_M0..M3 ごとに置かれ、各マスタには security mode を設定する**。APM_MASTERS_TEE は HP CPU と GDMA 暗号マスタ (AES / SHA) (82-84 行)、apm_hal_set_master_sec_mode は TEE 集合を TEE モード、APM_MASTERS_REE (それ以外) を REE0 モードにする (248-250 行)。**MODEM は 53-77 行のマスタ ID 表にも APM_MASTERS_* の定義にも明示項目が無い**。125-126 行のコメントは「この領域設定が無いと REE SRAM が MODEM マスタから見えず、Wi-Fi 初期化で APM 違反になる」と述べるのみで、REE 領域設定が modem のアクセスに影響し得るという示唆に留まる。**未確認**: modem の APM 上の識別 (どのマスタ ID か)、制御可能か、到達範囲、遮断できるか、policy が実機で効くか。TEE の例は設計の手掛かりであって Shizuku の起動所有権の証明ではない。
- 旧文書の断定の取り消し**: 「DMA により PMP 境界は無力化される」「真の隔離は成立しない」は、(a) blob を TCB に入れる設計では隔離が目的ではなく、(b) CPU 以外のマスタを制御する APM が存在するため (modem の扱いは未確認)、根拠不足として取り消す。逆に「APM で保護できる」とも主張しない。

未確認事項 (実証に必要な作業): modem DMA を APM の領域・経路で遮断できるか (TRM の APM 章は未読)、PMP と APM の設定を IDF 起動コード無しで Shizuku が所有できるか、policy が違反を実機で遮断することの実証。いずれも未実施。なお Shizuku 側の STREAM_CONNECT (object_api.hpp:98-102) はカーネルが DMA チャネルを握り MPU を素通りするためオブジェクトへ渡さない、という設計で、SDK 境界での DMA policy の考え方の参考にはなるが、C6 の無線 DMA への適用は未検討。

8. XNO への配置の妥当性 (境界の観点。XNO リポジトリは未読)

分担の前提: Shizuku = チップ固有 SDK・toolchain でカーネル / ペリフェラル / イメージを担当。XNO = SDK 無しの ISA 別 freestanding object。

OSI の部分	配置案	理由と未確認点
キュー、セマフォ、ミューテックス、イベントグループ、イベントキュー (NPL)、コールアウトの データ構造と待機意味論	XNO 候補 (条件付き)	チップ非依存に書ける。ただし待機・起床・タイムアウトは Shizuku のカーネルオブジェクトの機構 (今は無い) を呼ぶ必要があり、その ABI は object_api.hpp (今は内部ヘッダ。公開 ABI へ移す予定とヘッダにある) に依存する。blob からの 同期呼出し頻度・遅延 に対しオブジェクト間呼出し (CALL_METHOD) が耐えられるかは 未測定 で、旧文書の「毎秒数千〜数万回」は根拠が無く削除した。XNO リポジトリの toolchain が RV32IMAC を生成できるかも未確認。
割込み割当、ISR 登録、clock、PHY、modem リセット、MAC/eFuse、RNG、RegDMA、sleep retention	Shizuku (チップ固有)	C6 のレジスタ・割込みマトリクス・PMU を直接扱う。XNO が SDK を持たない境界からは置けない。
wifi_osi_funcs_t / npl_funcs_t / ext_funcs_t の 表そのものとその版・条件付きフィールド	Shizuku (チップ固有)	表のレイアウトは target 条件 (CONFIG_IDF_TARGET_*) と _version に依存し、blob のバイナリ互換契約。チップ別なので Shizuku 側の image に置く。表の各エントリが指す先を XNO オブジェクトの呼出しにするか Shizuku 内の関数にするかは設計選択。
blob 本体 (libnet80211.a 等)	Shizuku (チップ固有)	chip 固有バイナリ。XNO 側 freestanding toolchain とは別 toolchain / ABI。ABI 整合 (-march / -mabi、-ffreestanding 等) は未検証。
メモリ確保 (_malloc*、_wifi_*alloc)	Shizuku	DMA 可否・SRAM 区分は chip 固有で、保護 (PMP/APM) と arena 設計に結びつく。

妥当性の判断: 「OSI object を XNO に置く」は **OS 意味論の部分に限り設計上妥当で、全体を置くのは境界に反する**。blob の保護問題を理由にした全面的な「成立不能」は取り消す (第 7 節)。性能は未測定。

9. Bazel 静的 kernel.a + ESP-IDF 最終リンク案

旧文書は「FreeRTOS を起動しない条件と ESP-IDF の Wi-Fi/BLE リンクは両立しない」と断定していたが、リンク解決の事実は確認していないため取り下げる。確認できているのは次のとおり。

- ESP-IDF の標準経路では、app_main は FreeRTOS が起動した後に main_task から呼ばれる (旧文書の記述。本書では再確認していない [未確認])。
- ext_funcs_t の _task_create と npl_freertos_funcs_init は IDF の FreeRTOS 実装を前提にした実装で (bt.c:1098、npl_os_freertos.c)、npl_funcs の差し替え方・esp_register_ext_funcs (1091) / esp_register_npl_funcs (1105) に渡す表を自前にすることは構造上可能に見える。ただし BLE 初期化は表の登録に留まらず、modem_clock_module_enable、esp_phy_modem_init、coex_init、esp_ble_register_bb_funcs (bt.c:1120-1143) など IDF 内部の呼出しを含む (第 4 節)。**ただしその経路を実際にリンク・実行して確かめてはいない。**
- 標準の IDF 初期化 (PHY、clock、NVS、イベントループ) が FreeRTOS 型の API を必須で呼ぶ範囲は未確認。
- ポート計画書 (docs/06_esp32_port_manifest.typ:38,43) は、無線を残すなら FreeRTOS を残して Shizuku を 1 つのタスクとして動かす案から始めるよう推奨していた。この推奨自体は本書で否定していない。

10. 未確認事項 (実証不能なものを推測で埋めない)

- commit SHA と原文行番号**: IDF v6.1 は fff9895c82d744c7237be8847347bdd1b07c6643 で確定。**NuttX master と esp-hal は SHA 未固定で、WebFetch 要約の行番号は要再確認**。docs/06 が固定する IDF v6.1.1 は未照合。
- OSI 表の blob 側の実際の使われ方**: どの関数をどの頻度・どの文脈 (ISR か) で呼ぶか。優先度継承やタイマコールバックの実行文脈が必須か。
- 時間制約**: Wi-Fi/BLE の slot 境界・IFS・ACK 応答を、Shizuku の GRANT/スケジューラ構成で満たせるか。実機未検証。
- APM**: modem DMA を APM で遮断できるか、policy の実効、Shizuku による所有 (第 7 節)。
- NULL エントリを blob が呼ぶか**: Wi-Fi の _coex_condition_set、BLE の _coex_wifi_sleep_set / _coex_core_ble_conn_dyn_prio_get / hw_set_isr (第 3-4 節)。
- C6 の SRAM 区分**: DMA 可能領域と _malloc_internal の要求。
- PHY・clock・sleep retention を FreeRTOS 抜きで再利用できる範囲**。
- Zephyr / NuttX / esp-radio の各アダプタが blob に対して実際にどの機構を使っているか** の全体把握 (NuttX は nxsem / pthread_mutex / mqueue (file_mq_*) / kthread_create を使うという要約のみ)。
- XNO 側**: toolchain の出力形式、同期呼出しコスト、公開 ABI の確定時期。
- OSI 表の将来版**: _version が上がった場合の追従コストは未知。

11. 参照箇所一覧

対象	参照位置	内容
カーネル ABI	internal_headers/shizuku/kernel_abi.hpp:28-44	プリミティブ CALL RETURN SWITCH GRANT。

対象	参照位置	内容
オブジェクト API	internal_headers/shizuku/object_api.hpp:15-138 (SPAWN 46、YIELD 49、RUN_FOR 58、SLEEP_US 65、 SET_BUDGET 70、メモリ 75-78、STREAM 94-102、 KILL_THREAD 123)	公開済みの番号。同期プリミティブ・タイ マの番号は無い。
スレッド状態	internal_headers/shizuku/templates/ thread.hpp:77-78	優先度・起床時刻は方針側が持つ旨。
カーネル本体	internal_headers/shizuku/templates/ kernel.hpp:144-165,187,259	spawn_request、svc_dispatch、 timer_expired。
強制回収	source/kernel/thread.cpp:351、source/kernel/ dispatch.cpp:347	期限による取り上げ。
スケジューラ・sleep	source/kernel_object/handler.cpp:713-824、 827-862、980	回転子選択、待機ループ、table_lock。
メモリ	internal_headers/shizuku/templates/ kernel_object.hpp:111-138	ブロック構造と 24 階級。
C6 arch / board	internal_headers/shizuku/archs/rv32_c6.hpp:14,20、 boards/esp32_c6.hpp:14,19、ports/esp32/c6/ pmp.hpp:8-16	未完成 (fail-closed)、16 PMP エント リ。
ポート計画書	docs/06_esp32_port_manifest.typ:15-17,38,43	C6 の障害整理と推奨。
IDF Wi-Fi OS	wifi_os_adapter.h (v6.1)	wifi_osi_funcs_t。
IDF BLE	bt.c (v6.1) の 87-94、96-114、523-548、 1091-1145、npl_os_freertos.c	osi_coex_funcs_t、ext_funcs_t、 npl_funcs。esp_adapter.c:628-758 に Wi- Fi の C6 初期化子 (126 エントリ)。 APM は esp_tee_apm_prot_cfg.c と apm_defs.h:20。
NuttX	esp_wifi_adapter.c (master)	g_wifi_osi_funcs (720-847、127 項目、 二次抽出)。
esp-radio docs	1.0.0-beta.2、0.18.0、rtos-driver	スケジューラ要件と RTOS 契約。
IDF C6 セキュリティ	security.html	メモリ保護 API は private。

※本書は設計検討書であり、新規の自動テストコードは無い。上層が監査する「テストコード file:line」は該当なし。